Identifying the LLM Family Behind a Chatbot Response

Zihao Zhou (team leader and only member)

CMPSC 448 Midterm Project

October 11, 2026

Code: https://github.com/Dareenn09/cmpsc448-midterm

Abstract

We study whether a neural network can tell which LLM wrote a chatbot answer. We test this on 8,116 answers to the first prompt of a conversation from the LMSYS Chatbot Arena, split evenly across the Claude, GPT, Gemini and Qwen model families. We train a CNN and an LSTM and give them the prompt, the answer, or both. For RQ1, the answer alone is often enough to identify the family. The CNN reaches 0.654 accuracy and the LSTM 0.567, while random guessing gives 0.25 and a TF-IDF baseline with logistic regression reaches 0.592. For RQ2, the prompt carries almost no useful information. On its own it gives 0.27 to 0.29, and adding it to the answer does not help the CNN, which reaches 0.655, and hurts the LSTM, which falls to 0.525. For RQ4, we look at writing style to see what the classifier relies on. The families format their answers differently, for example Claude uses more bullet lists and Gemini more bold text. A classifier that sees only 16 style features still reaches 0.522. When we remove all formatting from the answers, CNN accuracy drops from 0.654 to 0.576. Formatting is a large part of the signal, but not all of it, because the families also differ in word choice.

Identifying the LLM Family Behind a Chatbot Response

Different LLMs are trained on different data and tuned in different ways, so their answers may carry a recognizable style. We test whether a classifier can learn this style. We frame the problem as text classification with four classes. Each example is a (LLM_name, LLM_Input, LLM_output) triple, and the label is the family of LLM_name. We answer three of the project's research questions:

- **RQ1:** Can a CNN and an RNN identify the LLM family from the response alone?
- **RQ2:** Does the user's prompt help? We compare models that see only the prompt, only the response, or both.
- **RQ4:** What characteristics distinguish the families? We look at length, vocabulary, lexical diversity, formatting and phrasing, and we remove kinds of signal from the responses to see how much accuracy drops.

Data

Source

We used the public LMSYS Chatbot Arena human preference dataset, lmsys-arena-human-preference-55k (Chiang et al., 2024). In Chatbot Arena, a user writes a prompt, two anonymous models answer it, and the user votes for the better answer. The file has 57,477 such battles, each with the prompt, the two model names and the two responses.

Building the Dataset

The script prepare_data.py turns the raw file into rows with the columns LLM_name, LLM_family, LLM_Input and LLM_output. LLM_family is derived from LLM_name and is the label we predict. The script works in the following steps.

1. Conversations can have several turns. We kept only the first prompt and the first response of each side, because later turns depend on earlier ones.
2. Each battle gives two rows, one per model, with the same prompt.
3. Model names were mapped to families by substring, so any name containing "gpt" counts as GPT. Rows with an empty prompt or response were dropped.
4. We kept the four families Claude, GPT, Gemini and Qwen and sampled the same number of rows from each. The smallest family, Qwen, has 2,029 usable rows, so every family has 2,029 rows and the dataset has 8,116 rows. Balancing makes accuracy easy to compare with the chance level of 0.25.

Table 1 lists the model versions in each family. We predict the family rather than the exact model name, because versions in the same family often write in a similar way and several versions have fewer than 100 rows. LLM_name is never given to the classifier.

Table 1

Model Versions in the Dataset

Family	Versions
Claude	claude-2.1 (676), claude-1 (523), claude-instant-1 (504), claude-2.0 (326)
GPT	gpt-3.5-turbo-0613 (440), gpt-4-1106-preview (436), gpt-4-0613 (431), gpt-4-0314 (270), gpt-3.5-turbo-1106 (215), gpt-3.5-turbo-0314 (78), gpt-4-0125-preview (77), gpt-3.5-turbo-0125 (57), gpt4all-13b-snoozy (25)
Gemini	gemini-pro-dev-api (1,046), gemini-pro (983)
Qwen	qwen-14b-chat (1,070), qwen1.5-72b-chat (551), qwen1.5-7b-chat (208), qwen1.5-4b-chat (200)

Note. The number of rows for each version is given in parentheses.

Responses have 175 words on average, with a median of 158 and a maximum of 897. There are 7,423 different prompts. About 10% of rows share their prompt with a row from another family, because both models in that battle belonged to our four families.

Splits

The script split_data.py makes a 70/15/15 split stratified by family with random seed 42. The split is by row, not by prompt, so 11.7% of test rows have a prompt that also appears in the training set.

Models and Training

Text Preprocessing

We split text into words, punctuation marks and line breaks with a simple regular expression. Each line break becomes a token <nl>. We kept capital letters and markdown symbols such as #, * and -, because they are part of a model's style. The vocabulary comes from the training set and keeps tokens that appear at least twice, 30,105 for responses, and the rest become <unk>. Each example is cut to 400 tokens, which is enough for most responses.

Input only and Output only use just the prompt or the response tokens. For Input + Output, the input is up to 100 prompt tokens, a separator token <sep>, then response tokens up to the limit of 400 tokens. We capped the prompt so that a long prompt cannot push the response out.

CNN

Each token is first turned into a vector of size 128, which the model learns during training. A convolution layer with 128 filters and ReLU then looks at every window of 5 tokens in the text. Each filter learns one short pattern, and max pooling keeps only its strongest match anywhere in the text. These 128 values go through dropout of 0.5 and a linear layer that gives a score for each of the four families. We tried filter widths from 2 to 5 and kept width 5 because it had the best validation macro F1, which was 0.615, 0.649, 0.654 and 0.657 for widths 2 to 5. The model has 3.94 million parameters, and almost all of them are in the token vectors.

RNN (LSTM)

The RNN uses the same token vectors of size 128, with dropout 0.5, followed by an LSTM with one layer, one direction and 128 hidden units. Sequences are packed so that the LSTM skips padding. The final hidden state, which has read the whole text, goes through dropout and a linear layer. The model has 3.99 million parameters.

Training

Both models are trained the same way:

- Cross entropy loss, Adam optimizer with learning rate 0.001, batch size 64, gradient clipping at norm 5.
- At most 20 epochs. After each epoch we compute macro F1 on the validation set. Training stops when it has not improved for 4 epochs, and the weights from the best epoch are restored.
- The test set is evaluated once, with the restored weights.
- Every setting is run with three random seeds, 42, 1 and 2. The data split stays the same, and the seed only changes the initial weights and the batch order. We report the mean and standard deviation over seeds.

Baseline

As a reference we trained a logistic regression with C = 5 on TF-IDF features of word unigrams and bigrams. It keeps capital letters, uses at most 50,000 features, and is run on the same splits and input settings as the neural models.

Results for RQ1 and RQ2

Table 2 gives the test accuracy and macro F1 of every model and input setting, where chance is 0.25.

Table 2

Test Accuracy and Macro F1 for Each Model and Input Setting

Model	Input	Accuracy	Macro F1
CNN	Input only	0.281 (0.015)	0.277 (0.018)
CNN	Output only	0.654 (0.012)	0.658 (0.013)
CNN	Input + Output	0.655 (0.003)	0.654 (0.006)
LSTM	Input only	0.271 (0.002)	0.270 (0.002)
LSTM	Output only	0.567 (0.022)	0.564 (0.027)
LSTM	Input + Output	0.525 (0.017)	0.525 (0.025)
TF-IDF + LR	Input only	0.286	0.285
TF-IDF + LR	Output only	0.592	0.592
TF-IDF + LR	Input + Output	0.568	0.568

Note. Values are the mean (standard deviation) over three seeds. The baseline does not depend on the seed, so it has no standard deviation.

RQ1: Identifying the Family From the Response

On the response alone, the CNN reaches 0.654 accuracy and the LSTM 0.567, both far above the chance level of 0.25. The CNN also beats the TF-IDF baseline at 0.592, but the LSTM does not. The baseline is strong, so much of the signal is in which words and symbols appear. The CNN likely does better because it looks at five tokens at a time instead of two and sees line breaks and markdown symbols, which the baseline ignores.

Table 3 and Figure 1 break the results down by family. Claude is the easiest family for both models, and the CNN gets 0.823 of Claude responses right. GPT is the hardest, at 0.534 for the CNN and only 0.296 for the LSTM. Most mistakes are between GPT and Qwen. The CNN labels 27% of GPT responses as Qwen and 30% of Qwen responses as GPT. The LSTM labels almost half of the GPT responses as Qwen.

Table 3

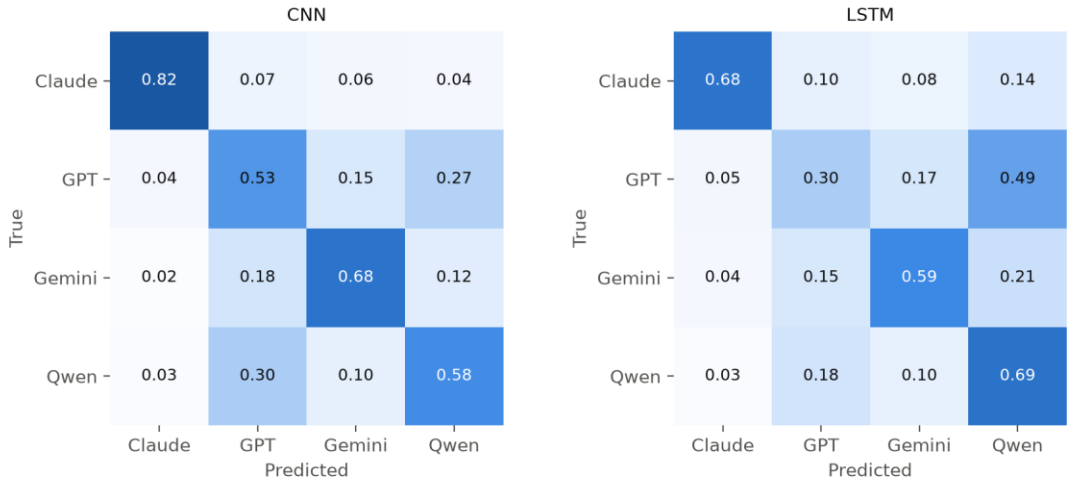
Test Accuracy for Each Family

Model	Claude	GPT	Gemini	Qwen
CNN	0.823 (0.035)	0.534 (0.051)	0.684 (0.027)	0.576 (0.004)
LSTM	0.684 (0.028)	0.296 (0.081)	0.592 (0.028)	0.694 (0.030)

Note. Models see the response only. Values are the mean (standard deviation) over three seeds.

Figure 1

Confusion Matrices of the CNN and the LSTM on the Test Set



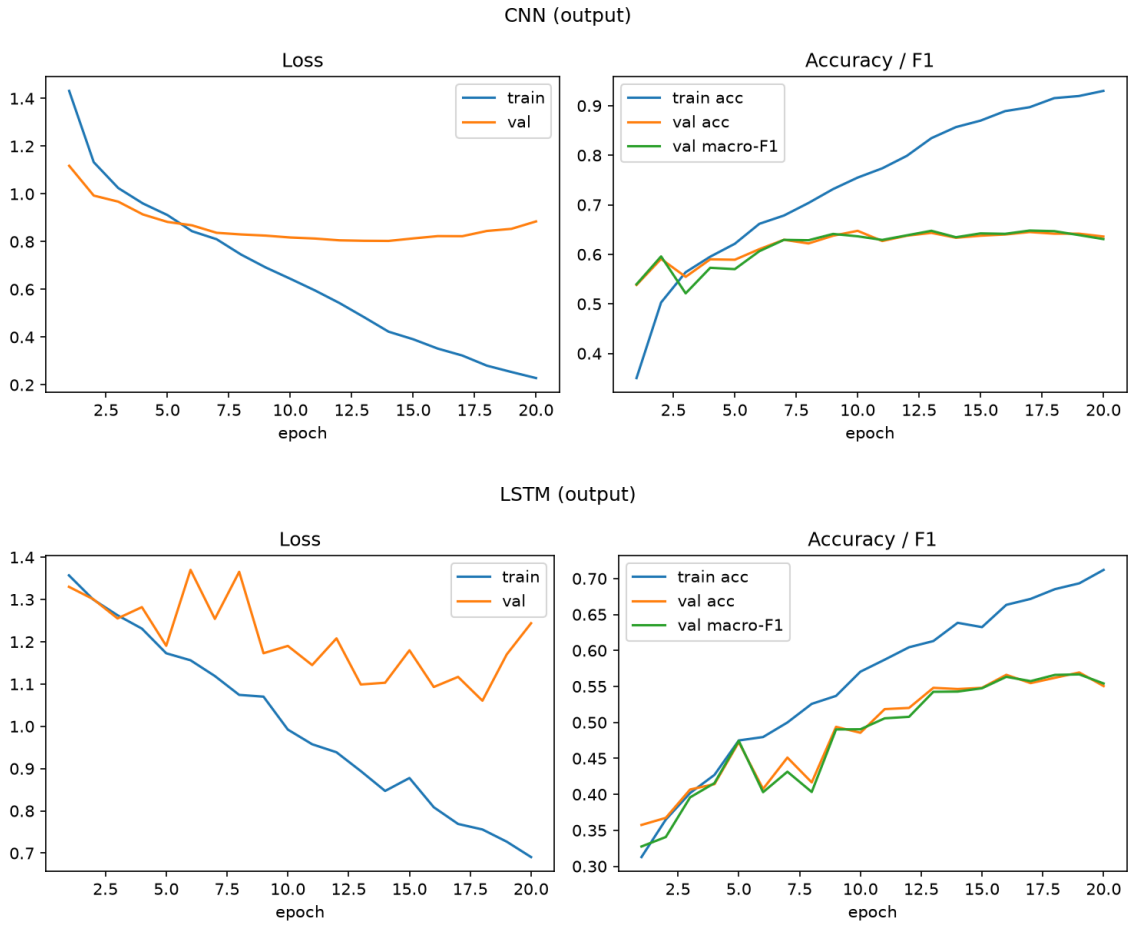
Note. Counts are summed over three seeds, and each row is divided by its total.

We think the CNN does better than the LSTM because the useful signals are short local patterns, such as "Here is", "* *" or a line break followed by a bullet. A CNN filter with max pooling finds such a pattern anywhere in the text. The LSTM has to carry this information to its final hidden state across up to 400 tokens, which is harder to learn from 5,681 examples.

Figure 2 shows the training curves for seed 42. The CNN overfits quickly. Training accuracy reaches 0.93, while validation accuracy stays near 0.64 after epoch 9 and validation loss starts to rise after epoch 14. The LSTM learns more slowly and less smoothly. In the three runs on the response alone, its best validation F1 came at epoch 19, 18 and 20 out of 20, so a longer training budget might have helped it.

Figure 2

Training Curves of the CNN and the LSTM



Note. Both models see the response only. The runs use seed 42.

RQ2: Does the Prompt Help?

Input only. The prompt alone gives accuracy close to chance, between 0.27 and 0.29 for all three models (Table 2). This is expected, because in Chatbot Arena the user writes the prompt before knowing which models will answer, so the prompt says almost nothing about the model. The small gain above 0.25 may come from prompts like "Hello" that appear in both the training and the test set.

Input + Output. Adding the prompt does not help. The CNN stays the same at 0.655 compared with 0.654, while the LSTM drops from 0.567 to 0.525 and the baseline from 0.592 to 0.568. Since the prompt carries almost no information about the family, it mostly adds noise. For the LSTM there are two more costs. The prompt takes up to 100 of the 400 tokens, so less of the response is seen, and the LSTM has to read through the prompt before it reaches the response.

The CNN is less affected because max pooling can simply ignore prompt patterns that are not useful.

RQ4: What Distinguishes the Families?

To see what makes the families different, we first compare simple style features such as length and formatting. We then train a classifier on these features alone, look at which words a simple classifier relies on most, and finally retrain the CNN after removing parts of the responses.

Style Features

We computed 16 features for each response in four groups: length, lexical, formatting and phrasing. Lexical diversity is measured with MATTR (Covington & McFall, 2010). Some features come from the assignment, and others are habits we noticed while reading the responses. Table 4 shows the features that differ most between families, and the full list is in rq4_features.py.

Table 4

Style Features for Each Family

Feature	Claude	GPT	Gemini	Qwen
Length				
Median words	165	145	185	147
Words per sentence	16.29	17.33	14.67	19.45
Lexical				
Lexical diversity (MATTR)	0.80	0.79	0.78	0.77
Formatting				
Line breaks per 100 words	6.09	3.93	5.58	2.64
% with bullet list	41	10	31	7
% with numbered list	16	30	31	26
% with bold text	0	10	35	6
Phrasing				
% opening with Sure / Certainly	0	8	1	6
% containing Chinese characters	1	1	1	8

Note. Length and line breaks are medians.

The families differ mostly in formatting, while length and lexical diversity are similar. Claude uses many bullet lists but no bold, Gemini uses the most bold text, and Qwen mostly writes plain paragraphs. Claude is almost the only family that uses the • character for bullets, and 12.6% of its responses contain "So in summary", a phrase the other families never use.

A logistic regression on one feature group at a time reaches 0.263 with length, 0.295 with lexical, 0.319 with phrasing and 0.443 with formatting features. All 16 together reach 0.522

(0.511 without the Chinese character feature, which we added after seeing the top words), still well below the CNN at 0.654, so the CNN must also use specific words.

The words with the largest weights in the TF-IDF logistic regression agree with Table 4. Beyond formatting symbols, GPT is marked by words such as "Certainly", "various" and "assist", and Qwen by the Chinese comma and common words such as "and" and "which", which may explain why it is often confused with GPT.

Removing Signals From the Response

To see which signals the CNN depends on, we retrained it on three changed versions of the responses: with formatting removed (markdown symbols deleted and line breaks turned into spaces), with words hidden (every word replaced by the same token), and with only the first 50 tokens.

Table 5

CNN Test Accuracy After Removing One Kind of Signal

Input to CNN	Accuracy	Claude	GPT	Gemini	Qwen
Full response	0.654 (0.012)	0.823	0.534	0.684	0.576
Formatting removed	0.576 (0.023)	0.800	0.423	0.591	0.490
Words hidden	0.591 (0.011)	0.742	0.480	0.565	0.576
First 50 tokens	0.601 (0.010)	0.814	0.508	0.597	0.487

Note. Accuracy is the mean (standard deviation) over three seeds. The family columns give the mean accuracy for each family.

Each change lowers accuracy by 0.05 to 0.08, but it stays far above chance. Removing formatting hurts the most, mainly for GPT and Gemini, while Claude can still be recognized by its wording. With all words hidden the CNN still reaches 0.591, about the same as the TF-IDF baseline, so layout and punctuation alone carry a lot of information. The first 50 tokens give 0.601, so much of the signal is already in how a response starts.

Limitations and Lessons Learned

Limitations

Data.

- The data covers older model versions only, Claude 1 and 2, GPT-3.5 and GPT-4, Gemini Pro, and Qwen 14B and Qwen1.5. Newer models may have different styles, so our numbers may not hold for them. Each family also mixes several versions, for example GPT-3.5 and GPT-4, which makes it harder to learn as one class.

- Our family mapping uses substrings of the model name. While writing this report we found that it labels gpt4all-13b-snoozy as GPT, although it is not an OpenAI model. These are 25 of the 2,029 GPT rows, or 1.2%. The effect on the results should be small, but the mapping should use an explicit list of names.
- Balancing to 2,029 rows per family discards most GPT and Claude data. With more data the models, especially the LSTM, would probably do better.
- We used only the first turn of each conversation and a single 70/15/15 split.

Models and training.

- We tuned only the CNN filter width. Other settings such as embedding size, dropout, learning rate and maximum length were fixed.
- Three seeds give a rough idea of the variance, but some differences, such as the one between the CNN with and without the prompt (0.654 and 0.655), are smaller than one standard deviation and should be read with care.

RQ4 analysis.

- When counting sentences, we treat "1." in a numbered list as the end of a sentence, so words per sentence is too low for responses with long numbered lists.
- List detection only recognizes lines starting with -, *, • or +, so some lists may be missed.
- We tested the changed responses only with the CNN, not with the LSTM.

Lessons Learned

1. A simple baseline is worth running first. TF-IDF with logistic regression trained in seconds and already reached 0.592, which gave us a clear target for the neural models.
2. Normal text cleaning would have hurt us. Lowercasing and removing punctuation and line breaks would delete the formatting that separates the families, and removing formatting alone lowered accuracy by about 0.08.
3. One run is not enough. The same LSTM gave anywhere from 0.542 to 0.584 with different seeds, so we report the mean and standard deviation over three seeds.
4. We used the test set only at the end. We chose the filter width and the stopping epoch on the validation set, and kept width 5 even though width 4 was slightly better on test, with 0.659 accuracy against 0.654.
5. Looking at the data by hand was useful. Listing the model versions showed a model that was put in the wrong family, and reading responses gave us several of the style features.

Conclusion

A CNN and an LSTM can tell which LLM family wrote a response much better than guessing. For RQ1, the CNN reaches 0.654 and the LSTM 0.567, while guessing gives 0.25. For RQ2, the prompt alone is close to guessing, and adding it to the response does not help. For RQ4, the families differ mostly in formatting and a few typical phrases, and removing either formatting or words lowers accuracy by 0.05 to 0.08. GPT and Qwen are the hardest pair to tell apart.

References

Chiang, W.-L., Zheng, L., Sheng, Y., Angelopoulos, A. N., Li, T., Li, D., Zhang, H., Zhu, B., Jordan, M., Gonzalez, J. E., & Stoica, I. (2024). *Chatbot Arena: An open platform for evaluating LLMs by human preference* (arXiv:2403.04132). arXiv. https://doi.org/10.48550/arXiv.2403.04132

Covington, M. A., & McFall, J. D. (2010). Cutting the Gordian knot: The moving-average type-token ratio (MATTR). *Journal of Quantitative Linguistics, 17*(2), 94–100.